

QID	Question Name	Des	Tag	Solution	Key Idea	From	Note
	Classrooms	校园里有 n 个拟举办的活动和 k 个教室。给定每个活动的具体开始时间 s_i 和结束时间 t_i , 同一个教室内的活动不能有任何时间重叠 (即使是前一个的结束时间等于后一个的开始时间也不行), 求在这 k 个教室中, 最多能安排多少个活动	greedy, sorting, multiset	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%202B.%20Classrooms/B.%20Classrooms.cpp	贪心策略一 (Earliest End Time): 将所有活动按结束时间升序排序, 优先处理结束早的活动, 为剩下的活动留出更多时间。 贪心策略二 (Latest Valid Classroom): 为活动分配教室时, 将其放入 $1-k$ 个活动结束时间最晚且当前可用的教室中, 通过最小化前一个活动结束后与后一个活动开始之间的“时间缝隙 (gap)”, 可以使相同数量的教室容纳更多的活动。	w2 ps2 b	如果使用普通的 for 循环遍历寻找最晚空闲的教室, 会导致 Time Limit Exceeded (TLE) 超时问题。 正确的优化方法是使用 std::multiset 来维护各个教室当前最后一个活动的结束时间。 每次利用 lower_bound() 找到第一个结束时间大于等于当前活动开始时间的迭代器, 然后将其向前移一位 (即 prev(iterator)), 就能高效找到满足条件且结束时间最晚的教室。
	Array Division	给定一个正整数数组, 判断能否在最多移动一个元素 (删除并插入到任意位置) 后, 将数组分割为两个元素总和相等的连续非空部分 (前缀和后缀)。	prefix sum, multiset, math	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%202D.%20Preparing for the Contest/20for%20the%20Contest.cpp	维护前缀和后缀集合: 若数组总和为奇数直接 NO, 使用两个 multiset 记录前缀和后缀元素, 遍历数组更新 current_sum 并分3种情况: 1. 若等于 half, 直接 YES; 2. 若小于 half, 检查后缀集合中是否存在值为 (half - current_sum) 的元素可移至前缀; 3. 若大于 half, 检查前缀集合中是否存在值为 (current_sum - half) 的元素可移至后缀。	w2 ps2 c	1.使用 long long 防止溢出; 2. multiset 删除单个元素需使用 erase (find(val)) 以防删除所有重复值; 3. 题目允许原地地移动, 故原本就满足条件的情况直接输出 YES。
	Preparing for the Contest	有 m 个不同复杂度的 bug 和 n 个不同能力及报酬的学生, 学生只能修复复杂度不超过自身能力的 bug, 求在总报酬不超过 s 的前提下, 修完所有 bug 所需的最少天数, 并输出分配方案。	greedy, binary search, priority queue	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%202D.%20Preparing for the Contest/20for%20the%20Contest.cpp	二分答案 + 贪心 (优先队列): 1. 二分天数: 天数具有单调性, 范围 1 到 $m, 2$ 分组打包; 在 check(x) 中, 每人最多修 x 个 bug, bug 按复杂度排序, 每个人打成一个包; 2. 优先队列选人: 将所有能力大于等于该包最大难度的学生放进优先队列 (按报酬构成的最小堆), 每次弹出堆顶 (当前最优的边标学生); 包超过 x 个 bug, 4. 累加报酬若没超 s 则该天数可行, 向左逼近。 离线处理+逆向时光倒流, 正向用并查集最难实现“删边”, 故先预处理得出所有操作执行完毕后的“终态”图并建立并查集, 正向遍历时利用 swap 巧妙在查询数组中缓存城市修改前的旧人口, 逆序处理查询: “删边”转为“加边(union)”, “修改人口”转为“恢复旧人口”, 使用 multiset 动态维护各连通块人口总和, 通过 “max_pops.rbegin()”实时 $O(1)$ 获取最大值, 合并与撤销时使用 erase(find(x)) 精确移除旧状态, 整体时间复杂度 $O(Q \log N)$ 。	w2 ps2 d	1.不能单用 multiset 技能力刚达标的人, 因为能力够但不一定便宜, 必须用优先队列维护性价比; 2.priority_queue 默认最大堆, 需自定义比较器实现最小堆; 3.二分结束后, 需用最终确定的最小天数再跑一次 check 函数来保存具体的分配方案; 4.打包遍历赋值时, 最后一组 bug 可能不足 x 个, 注意防止数组越界。
	Ancient Berland Roads	给定 n 个城市, m 条双向道路及初始人口, 执行 Q 次查询: 摧毁指定道路 (D) 或修改指定城市人口 (P), 求每次操作后全国最大连通块的人口总和。	DSU, offline queries, graph	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%202F.%20OpenStreetMap/apF.%20OpenStreetMap.cpp	二维滑动窗口最小值 (单调队列降维打击): 1. 暴力遍历会超时, 需将二维问题降维成两次一维滑动窗口; 2. 先对原矩阵的每一行使用长度为 k 的单调队列求最小值, 存入新矩阵 matrix_1; 3. 再对 matrix_1 的每一列使用长度为 k 的单调队列求最小值, 存入 matrix_2; 4. 再次处理后 matrix_2 中的元素即为每个 $k \times k$ 子矩阵的最小值, 全部累加即为答案; 5. 单调队列核心: 双端队列 (deque) 内存值元素的“表”可以判断队头是否滑出窗口 (过期), 入队前从队尾弹出所有大于等于当前元素的值, 严格维持队列单调递增。	w2 ps2 e	1. 利用 swap 缓存历史状态, 极大地简化了空间与逻辑。 2. 必须使用 multiset erase(multiset.find(val)) 进行精确删除, 若直接 erase(val) 会误删拥有相同人口总和的独立连通块。
	OpenStreetMap	给定一个 $m \times n$ 的高度矩阵, 要求计算其中所有大小为 $k \times k$ 的子矩阵的最小高度之和, 由于数据量大, 高度数据需先根据题目提供的公式通过递推生成。	monotonic queue, sliding window minimum, 2D array	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%202F.%20OpenStreetMap/apF.%20OpenStreetMap.cpp	二分答案 + +/- 1 映射 + 二维前缀和。 1. 难以直接维护滑动窗口的中位数, 转而二分猜测最小中位数 mid; 2. 核心 check(mid) 函数: 将原矩阵中小于 mid 的数映射为 -1, 大于的映射为 1, 等于的映射为 0; 3. 利用二维前缀和 prefix 求出每个 $H \times W$ 窗口的和 sum; 4. 若存在 sum <= 0, 说明该窗口真实中位数 < mid, 缩小右边界 (hi = mid); 若所有窗口 sum > 0, 扩大左边界 (lo = mid + 1), 整体复杂度 $O(1)$ 。	w2 ps2 f	1. 防止数据污染: 强烈建议全面采用 0-indexed (从 0 开始索引) 构建矩阵, 防止占位符 0 变成最小值污染答案。 2. 二维数组赋值陷阱: C++ 中 vector<vector> 是按行存储的, 将操作得出的一维结果赋值回矩阵时, 必须用内层循环逐个填入 (matrix[i][c] = col_min[i]), 绝不能直接按列赋值, 否则会导致越界崩溃。 3. 防溢出与精度: 高度公式不可使用递归 (会爆栈), 必须用迭代推导; 乘法计算极易溢出, 相关变量及矩阵必须全生命周期使用 long long, ((i-1) * m + j - 1) 是二维坐标到一维数组的映射公式 (假设 $S[0]$ 和 $S[S]$ 是从 1 开始的)。
	Quality Of Living	给定一个 $m \times n$ 的城市矩阵, 每个街区有唯一的质量排名 (1到 R^C), 求一个尺寸为 $h \times w$ (均为奇数) 的矩形子区域, 使其内部所有街区排名的中位数最小 (即数值最小)。	binary search, 2D prefix sum	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Workshop/Workshop%202/Quality%20of%20Living.cpp	二分答案 + +/- 1 映射 + 二维前缀和。 1. 难以直接维护滑动窗口的中位数, 转而二分猜测最小中位数 mid; 2. 核心 check(mid) 函数: 将原矩阵中小于 mid 的数映射为 -1, 大于的映射为 1, 等于的映射为 0; 3. 利用二维前缀和 prefix 求出每个 $H \times W$ 窗口的和 sum; 4. 若存在 sum <= 0, 说明该窗口真实中位数 < mid, 缩小右边界 (hi = mid); 若所有窗口 sum > 0, 扩大左边界 (lo = mid + 1), 整体复杂度 $O(1)$ 。	w2 wks q3	1. 单调性陷阱: 二分 check 的判断条件必须是 sum <= 0 而非 == 0, 否则会破坏二分的单调性导致方向找错。 2. 空间优化: +/- 1 映射可在计算二维前缀和时实时得出 (int curr = ..., 无需额外开辟二维数组来存储映射值。 3. Grader 评测机避坑: 此类交互题不可提交 main 函数: 函数参数签名必须严格匹配后台要求 (如原生数组 int q[3001]), 若写成 int* 会导致链接器报错 undefined reference。 4. 0-based 索引转换: 题目查询多为 1-based (第 Sx 到第 Sy 题), 传入查询和更新函数时必须统一减一。
	The Problem Set	给定 N 个题目的难度数组, 需要处理 5 种操作: 单点更新难度 (U)、查询区间最高难度 (M)、查询区间难度总和 (S)、判断区间是否按难度单调递增 (I)、判断区间是否按难度单调递减 (D)。	segment tree, point update, range query	https://github.com/hianzheyu-tesd/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%203/A.%20The%20Problem%20Set.cpp	1. 区间最大值、求和以及单调性都满足结合律, 可使用线段树 (Segment Tree) 维护。 2. 单调性维护核心: 线段树节点除了 max 和 sum, 必须额外记录首尾元素 (first_val 和 last_val) 以及单调性布尔值 (is_inc 和 is_dec)。 3. 区间合并逻辑 (Push Up): 父区间为单调递增的充分必要条件是“左子区间为单调递增”且“右子区间为单调递增”且“左区间的 last_val 小于或等于 右区间的 first_val”, 递减同理。	w3 ps a	1. 内存与无限递归防线 (std::length_error): 必须对输入进行合法性校验 (if ((l < cin >> n >> m)) 以及 if (n <= 0)), 否则当评测机输入为空或 S_N 远 S 时, 可能会读入负数导致堆栈溢出导致 vector 请求超大内存撑爆上堆, 或导致 build(0, -1) 陷入无限递归不断 push_back。 2. 跨区间查询合并陷阱 (致命逻辑错误): 在 search 跨越左右子树时, 合并首尾元素进行比较, 必须且只能使用左右子查询返回的结构体结果 (如 left_search.last_val), 严禁直接使用底层树节点的属性 (如 tree[left_index].last_val), 否则会将查询区间之外的脏数据纳入比较范围。 3. Push Up 全面性: update 后不仅要更新 max 和 sum, 千万别忘了把更新后的首尾元素 first_val 和 last_val 也要一路向上 Push Up。 4. 0-based 索引转换: 题目查询多为 1-based (第 Sx 到第 Sy 题), 传入查询和更新函数时必须统一减一。

QID	Question Name	Des	Tag	Solution	Key Idea	From	Note								
	Little Girl and Maximum Sum	给定包含 n 个元素的数组和 q 个区间求和查询。允许在查询前对数组进行重排。任务是找出使所有查询结果的总和达到最大的重排方案。并输出该最大总和。	difference array, greedy, prefix sum, sorting	https://github.com/hianzheyu-1ed/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%20SetC%201B.%203B.%202B.%201B.%20Girl%20and%20Maximum%20SumB.%201B.%20Girl%20and%20Maximum%20Sum.cpp	1.贪心策略:为了使总和最大,应将数值越大的元素放在被查询次数(频率)越多的位置上。将原数组和计算出的频率数组分别降序排序。对应相乘后累加即可。 2.差分数组高效统计:如果暴力遍历每个查询区间更新频率,时间复杂度为 $O(q \times n)$ 会超时。通过维护一个差分数组记录当前索引的增量变化。可将区间修改优化至 $O(1)$ 。即对于区间 $[a, b]$, 执行 $diff[a]++$ 且 $diff[b + 1]--$ 。 3.前缀和还原:所有查询处理完毕后,对差分数组求前缀和 ($prev[i] = prev[i - 1] + diff[i]$),即可在 $O(n)$ 内获得每个位置的最终查询频率。	w3 ps b	1.隐式整数溢出 (Integer Overflow): 频率 (最高 2×10^5) 与数组元素 (最高 2×10^5) 相乘时,极大可能突破 32 位 int 的极限。计算 result 时必须得将与乘法的变量强制转换为 64 位 (如 $(long\ long)/freq[i] \times freq \times (long\ long)array[i]$), 否则会溢出成巨大的负数。 2.差分数组右边界陷阱: 处理闭区间 $[L, R]$ 的差分时, 增益的消除发生在 $R + 1$ 处, 必须减去 $diff[R + 1]$ (代码中即 $diff[queries[i].second + 1]--$), 绝对不能在 R 处减。 3.前缀和越界防护: 计算前缀和还原频率时, 为避免 1-2 或类似逻辑导致数组越界 (Segmentation Fault), 建议引入额外的滚动变量 <code>current_freq</code> 累加差分值。 4.初始化: 差分数组的初始值必须为 0 而非 -1, 否则会污染后续所有的频率累加计算。								
	Xenia and Bit Operations	给定 2^n 个非负整数构成的序列。计算 v 的规则为: 第一次迭代对相邻元素进行按位或 (OR) 运算。第二次对新序列相邻元素进行按位异或 (XOR) 运算。两者交替进行直到剩下一个元素。题目要求在进行单点更新操作后, 实时输出新的总结果 v_n 。	segment tree, point update, bitwise operations	https://github.com/hianzheyu-1ed/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%20SetC%203C.%20Xenia%20and%20Bit%20OperationsC%20Xenia%20and%20Bit%20Operations.cpp	1.因为每一层由上一层的相邻元素计算得出, 所以非常适合使用线段树 (range tree)。 2.为了在 OR 和 XOR 之间切换, 在每个节点中记录当前高度 (height)。通过判断当前高度的奇偶性, 来决定当前层是执行按位或还是按位异或。 3.每次更新一个值时, 递归更新受影响的树节点 (即从叶子到根的路径)。时间复杂度为 $O(\log(2^n) \times \log n)$ 。最终结果 v 就是根节点的值。	w3 ps c	1.引用传递防爆线 (TLE/MLE 杀手): 在 C++ 中, 递归建树或更新时, 传递庞大的 vector 必须使用引用 (&)。若按值传递, 会导致每次递归都完整拷贝十几万大小的数组, 瞬间引发 Time Limit Exceeded 或 Memory Limit Exceeded。 2.整数运算陷阱: 计算 $S2^n \times n$ 的数组真实长度时, 严禁使用 <code><cmath></code> 中的 <code>round(pow(2, n))</code> , 因为 pow 依赖浮点数运算, 存在精度误差且常数巨大。计算 2 的次方应永远使用位运算 $1 \ll n$ 。 3.建树区间边界: 题目输入的 Sn 只是指数。线段树实际需要管理的区间长度是 $Sn + 1$ 以内。建树时传入的范围应当是 $[0, N - 1]$ 而非 $[0, n - 1]$ 。否则整棵树只会处理前十几个元素。 4.显式 vs 隐式线段树: 虽然带有 left, right, range 的显式结构体写法逻辑完美契合直觉。但在纯算法竞赛中, 面对固定长度的数组, 使用一维数组隐式线段树 (当前节点 S , 左孩子 $2S$, 右孩子 $S2+1$) 能极大节省内存 (几十 MB 缩减到几 MB) 并提升常数级别的运行速度。								
	Promotion Counting	给定一家由 n 头奶牛组成的树形结构公司。其中 1 号奶牛是总裁。每头奶牛都有一个独特的熟练度评分 (proficiency rating)。任务是帮每一头奶牛统计出, 在它的所有下属中, 有多少头奶牛的评分严格大于它自己的评分。	DFS order, segment tree, offline processing, point update, range query	https://github.com/hianzheyu-1ed/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%20SetC%203D.%20Promotion%20CountingD.%20Promotion%20Counting.cpp	1.公司组织结构为一棵多叉树, 无法直接套用二叉的线段树 (range tree)。 2.引入 DFS 序 (DFS order)。在 DFS 遍历时使用一个计时器记录每个节点被添加和移除的时间。从而将多叉树的子树范围完美映射为一维数组上的连续区间。 3.利用离线处理: 将奶牛按评分从大到小 (降序) 排序后两遍加入线段树。 4.每次处理时, 查询当前奶牛对应的 DFS 区间内的总和。因为按照降序加入, 此时线段树中已有的必定是评分更高的奶牛。所以查询结果即为所求的高分下属数量。	w3 ps d	1.DFS 时间戳防空窗: DFS 的计时器必须使用引用传递 (int& timer), 以确保分配的时间戳绝对连续 (1 到 n), 避免因值传递产生跳号, 从而导致线段树无效膨胀。 2.直线打乱顺序的映射: 按 rating 排序会彻底打乱奶牛原本 1 到 n 的编号顺序。必须使用额外的数组 (如 result[cows[i].index]) 按照奶牛的原始 index 存放查询结果, 以保证最终的输出顺序符合题意。 3.0-based vs 1-based 索引对齐: USACO 的输入数据通常从 1 开始 (如 manager 编号)。若代码底层使用从 0 开始的 vector 存储, 必须在读入时执行 -1 操作。否则会导致数据错位或树结构错误。 4.线段树查询的致命退化: 在 search 函数中, 只要当前节点的管辖区间与查询区间完全重合 (tree[root].range == range), 必须立刻 return 结果。若缺少此条件, 查询复杂度会从 $O(\log n)$ 严重退化为 $O(n)$, 直接导致 TLE。 5.递归扩容陷阱: 在基于 vector 动态 push_back 建树前, 务必使用 reserve(4 * n) 预先分配内存。否则在递归中一旦触发递归扩容, 会导致原有节点的引用失效, 引发难以排查的 Bug。								
	Points	在一个笛卡尔坐标系中处理三种操作: 1. add x y 添加点; 2. remove x y 移除点; 3. find x y 查找严格位于目标点正上方和正右侧的点。如果有多个符合条件的点, 优先选择最左侧的。若仍不唯一, 则选择最下方的点。	segment tree, coordinate compression, 2D spatial query	https://github.com/hianzheyu-1ed/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set/Problem%20SetC%203E.%20PointsE.%20Points.cpp	降维与坐标离散化: 将所有查询中出现的 x 坐标作为线段树的基础范围。因为最大 x 坐标可达 5×10^9 , 无法直接建这么大的树。所以收集所有出现的 x 坐标并去重, 映射到 $[1, M]$ 的 1-based 索引区间。 线段树节点设计: 线段树的每个节点维护其管辖 x 区间内的最大 y 坐标。在叶子节点 (代表具体的某个唯一的 x 坐标) 使用 set 来存储所有共享该 x 坐标的 y 值。 添加与删除操作: 每次操作时, 在叶子节点的 set 中插入或删除对应的值。然后沿该叶子节点到根节点的路径向上更新。如果删除后 set 为空, 则将最大 y 坐标重置为 -1。 带有弱枝的查找操作: 若当前节点的最大 y 坐标 <= target.y, 或当前区间完全在目标 x 的左侧 (range.second <= target_x_index), 说明子树中无有效点, 直接以 $O(1)$ 时间返回 $(-1, -1)$ 进行剪枝。为了满足“最左侧”的条件, 始终优先返回最左侧的叶子树。只有当叶子树返回 $(-1, -1)$ 时才去检查右子树。	w3 ps e	•暴力解法瓶颈: 暴力搜索的时间复杂度为 $O(n^2)$, 整体计算量会达到 $5 \times 10^4 \times 10^4$, 会导致超时 (TLE)。 •离散化三步曲: 使用经典的 <code>x.erase(unique(x.begin(), x.end()), x.end())</code> 来去除排序后的重复 x 坐标, 完成完美的空间压缩映射。 •防超时与防越界: 使用二分查找 (lower_bound/upper_bound) 代替 unordered_map 可以防止哈希碰撞导致的 TLE; 查找返回离散化索引后, 务必映射回原数组真实坐标后再输出。								

QID	Question Name	Des	Tag	Solution	Key Idea	From	Note
					1. 正则则反 (Reverse Thinking): "不能包含所有颜色"等价于"至少有一种颜色被完全排除"。我们可以枚举这 1 到 $K \times S$ 种颜色。假设当前排除颜色 C, 那么所有颜色为 C 的宝石就变成了绝对不能触碰的"地雷" (障碍物)。 2. 单调栈降维 (Monotonic Stack for Maximal Rectangles): 对于每种被排除的颜色, 问题转化为寻找"不包含地雷的最大空白矩形", 以两个相邻地雷的 X 距离为宽度, 地雷的 Y 坐标为限制高度。利用单调栈可以在 $O(N)$ 的时间复杂度内, 找出所有颜色对应的"极大候选矩形"集合。 3. 空间换时间 (Data Decoupling): 为了满足不同算法的需求, 宝石数据必须存两份。一份按颜色分组 vector<vector<Jewel>> 用于单调栈高效找矩形, 另一份的全局扁平 vector<Jewel> 用于后续的扫描线排序。 4. 扫描线 + 线段树 (Sweep Line & Segment Tree): 将所有宝石和找出的所有候选矩形统一按 Y 坐标 (高度) 从小到大排序, 用一根水平扫描线从下往上扫。遇到新高度 < 矩形高度的宝石, 就把它离散化后加入线段树维护。遇到新高度 > 矩形高度时, 直接在线段树中查询所有左右边界 (left, right) 的区间和, 即为该矩形能抓到的宝石总数。		1. 高度 -1 的极限值拉: 单调栈弹出的 height 是指你当前"地雷"的真实 Y 坐标。为了不把地雷抓进去, 矩形的有效抓取高度必须严格设为 height - 1, 否则答案会多算。 2. 漏算的"通天矩形": 单调栈只能抓到被顶部宝石卡住的矩形。如果两个地雷在 X 轴上相邻 (中间没地雷), 这片区域是可以无限往上抓的。必须在单调栈入栈前, 手动为每一对相邻的 X 坐标塞入一个高度为无穷大 (如 $S^2 \times 10^9 S$) 的"通天矩形"。 3. 哨兵的完美护航: 在跑单调栈之前, 必须在最左侧 ($SX=0S$) 和最右侧 ($SX=10^9+1S$) 加入两个高度为 0 的哨兵节点。同时, 如果后续用 unordered_map 搞离散化时, 必须把这两个哨兵的 X 坐标也提前加入离散化数组中, 否则会导致哈希表找不到 Key。 4. 线段树的致命退化: 在 search 函数的开头, 必须判断当前区间与查询区间是否完全重合 (if (tree[root].range == range))。如果写成左 endpoint 单独判断, 会导致查询无法及时 Return, 时间复杂度从 $O(N \log N)$ 瞬间退化到 $O(N^2)$ 导致 TLE。 5. pair 的比较陷阱: 在 C++ 中直接比较两个 pair 会默认先比较 first (X 坐标)。在单调栈中比较高度时, 必须明确写出 x[j].second < x[iq.back()][].second。
	Jewel heist	平面上有 n 颗宝石, 分为 k 种颜色。罗平使用一个机械手进行一次抓取。机械手会抓取某个水平线段及其正下方区域内的所有宝石。要求在"不能离开所有 k 种颜色"的前提下, 求出最多能抓取多少颗宝石。 ($N \leq 200,000$, 坐标最大值为 10^9)	monotonic stack, sweep line, segment tree, discretization, 2D range query	https://github.com/hianzheyu-test/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set%203/G.%20Jewel%20heist%20ness.cpp	w3 ps f		
	Restructuring Company	公司有 n 名员工 (初始各自为一个部门)。处理 q 次操作。操作分三种: 1. 合并员工 x 和 y 所在的部门; 2. 将编号在 $[x, y]$ 区间内的所有员工合并到同一个部门; 3. 查询员工 x 和 y 是否曾在同一个部门。 ($\$n \leq 200,000, q \leq 500,000$)	DSU, path compression, interval skip	https://github.com/hianzheyu-test/UNSW-COMP4128-Programming-Challenges/blob/main/Workshop%203/2.%20Restructuring%20Company.spe	w3 wks q2	• 巨大的查询量 (50万次) 要求必须使用 ios_base::sync_with_stdio (false); cin.tie(NULL); 来优化 I/O 速度。 • 如果只用一层 for 循环暴力进行 Type 2 操作, 必定会导致 TLE (超时)。 • 在 C++ 中传递 vector 时, 必须使用引用传递 (vector&int&), 否则对并查集的修改只会停留在局部副本中, 导致结果完全错误。	
					1. DFS序拍平: 用 DFS 遍历树, 记录每个节点进入和离开的时间戳 range.first 和 range.second。这把"子树"转化为了"数组上的连续区间"。把"祖先"转化为了"包裹该区间的更大区间"。 2. 线段树维护: 构建线段树维护区间是否"全满"。 3. 操作映射: - 填满 v (Type 1): 将区间 [v.first, v.second] 置为 1 (updateDown)。 - 清空 v (Type 2): 将单点 v.first 修改为 0 (updateUp)。 - 查询 v (Type 3): 查询区间 [v.first, v.second] 是否全为 1 (search)。 转化思想 (State Mapping): 线段树判断"满水"的条件是"整个子树区间全为 1"。因此, 清空 v 的祖先不需要一层层往上找, 只需要在子树 v 内部"戳"一个 (把单点置为 0)。那么所有包裹着 v 的祖先区间在查询时都会发现"内部不全为 1", 从而巧妙实现 O(log N) 清空祖先。 - 证明转移 (Evident Shift): 如果在 Type 1 填满子树时, 直接置置区间, 会移除掉之前为了清空祖先而留下的"洞 (0)"。因此必须在注水前检查: 如果当前区间全满, 那么 0 转移给 v 的直接父节点, 保全祖先的状态。		1. 致命陷阱 (Test 6 WA): Type 1 操作时, 必须先写 if (search(v) && parent[v] != 0) 进行状态转移 (updateUp(parent)), 然后再搞 v 注水 (updateDown(v))。否则反了或者漏了都会导致祖先被清空。 2. 线段树 Lazy Tag: 涉及区间覆盖, 必须严格在分区间前写 pushDown。在子节点更新完后写 pushUp。保证父子节点状态同步。 3. 数据量达到 50 万级别, 虽然你的代码能通过, 但加上 cin.tie(NULL); ios_base::sync_with_stdio(false); 能进一步防止 TLE。
	Water Tree	一棵 n 个节点的有根树 (根节点为 1)。三种操作: 1. 填满节点 v 及其所有子孙变满。 2. 清空节点 v 及其所有祖先变空。 3. 查询节点 v 利用 v 当前是否满水。(数据范围: $n, q \leq 500,000$)	DFS order, segment tree, lazy propagation, tree flattening	https://github.com/hianzheyu-test/UNSW-COMP4128-Programming-Challenges/blob/main/Workshop%203/3.%20Water%20Tree.cpp	w3 wks q3		
					1. 贪心局限性: 在第二列中, 选择局部最大值可能无法达到全局最大值。因此贪心算法无效, 必须使用 DP。 2. 状态转移 (自后向前): 由于不能连续选择牌, 状态转移方程为: $dp[i][j] = \max(dp[i][j+1] + 1, dp[i][j+2] + 1 + row[i] - 1, \text{height})$ 。 3. 剪枝逻辑验证: 为什么不需要考虑 $dp[i][j+3]$? 因为题目规定所有学生身高均为正整数 (>=1), 任何跨度达到或超过 3 步的跳跃在数学上都证明是次优选择。它总是可以被一系列更小的跳跃组合所替代, 从而产生严格更大的总和。		
	Basketball Exercise	存在 2 排学生。我们需要从 m 中选学生。使得身高总和最大。限制条件: 不能连续选择属于同一排的学生。并且挑选顺序从左到右。当前选中学生的索引必须严格大于前一个。数据范围: $n \leq 10^5, m \leq 10^9$ 。	DP	https://github.com/hianzheyu-test/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set%204/A.%20Basketball%20exercise2.cpp	w4 ps a	1. 数据溢出陷阱: 极值情况下总和会达到 10^{18} 级别, height 和 dp 数组必须声明为 long long。 2. Base Case 越界: 当 $n=1$ 时, 如果不做 $if (n == 1)$ 的特判, 在初始化 dp 等状态时会触发 Segfault。能进一步防止 TLE。	
					1. 状态定义: 利用 bitmask 记录已吃的菜品集合。同时记录上一道吃的菜以计算连击奖励。 $dp[mask][i]$ 表示已吃菜品集合为 mask 且最后吃的是菜 i 时的最大满意度。 2. Base Case: 如果只吃 1 道菜, 没有上一道菜, 满意度就是该菜的基础满意度。 3. 状态转移: 检查当前菜 i 是否在 mask 中。如果在, 则查看前驱 i 的前一个状态, 遍历前一个状态中最能吃到的菜 q, 并尝试加上 rule[q][i] 的奖励。取最大值填入当前 cell。 4. 结果统计: 遍历所有恰好包含 m 个元素的 mask, 取其中的最大满意度作为最终答案。		
	Kefa and Dishes	有 n 道菜。需要刚好点 m 道不同的菜以尝总满意度最大。每道菜有基础满意度。且有 k 条连锁规则: 如果刚吃完菜 x , 紧接着吃菜 y , 会获得额外的 z 点满意度。数据范围: $1 \leq m \leq n \leq 10^5$ 。	Bitmask DP	https://github.com/hianzheyu-test/UNSW-COMP4128-Programming-Challenges/blob/main/Problem%20Set%205/B.%20Kefa%20and%20Dishes.cpp	w4 ps b	1. 1-based 转 0-based 陷阱: 题目输入的规则 x 和 y 是从 1 开始的, 在存入规则数组 rules 时必须减 1, 否则会导致查找连击分数时错位。 2. 数据溢出: 满意度和热岛超过 int 上限, dp 数组、基础满意度和规则数组均需使用 long long。 3. 寻找 m 目标: 可以通过 __builtin_popcount() 快速判断 mask 中 1 的个数。 4. 性能表现: C++23 环境下 AC 耗时 328 ms, 内存 44900 KB。	

QID	Question Name	Des	Tag	Solution	Key Idea	From	Note														
	Coin Combinations II	给定 n 种面值的硬币和一个目标金额 x。求使用这些硬币凑出金额 x 的不重复组合数 (即不考虑硬币拿取的先后顺序。2+2+5 和 5+2+2 算作同一种方案)。结果对 1,000,000,007 (1e9 + 7) 取模。数据范围: 1 <= n <= 100, 1 <= x <= 1,000,000。	1D DP, Knapsack (完全背包)		1. 状态定义: dp[i] 表示凑出金额 i 的合法组合数。Base Case: dp[0] = 1 (凑出金额 0 只有一种什么都不拿的方案)。 2. 核心状态转移 (完全背包模型): 为了求“组合数”(消除顺序带来的重复), 必须强制硬币加入战场的先后顺序。具体做法是: 外层循环遍历硬币种类 (coin), 内层循环遍历目标金额 (i)。状态转移方程: dp[i] = (dp[i] + dp[i - coin]) % MOD。 3. 物理意义: 先用硬币 A 更新所有金额, 再引入硬币 B。这就保证了在任何合法方案中, 硬币 A 永远排在硬币 B 前面: 从根本上杜绝了 A+B 和 B+A 被重复计算的情况。	practice w4 q3	1. 循环顺序陷阱 (WA): 如果外层枚举金额 (i), 内层枚举硬币 (coin), 算出来的是“排列数”(Order matters)。这其实是上一道题 Coin Combinations I 的解法。做背包问题时, 求组合数必须外层枚举物品! 2. 内层循环起点优化: 内层金额循环可以直接从 i = coin 开始 (即 for int i = coin; i <= x; i++)。这样连 if (i - coin < 0) continue; 这句边界判断都省了。代码更干净且运行更快。 3. 常量定义雷区: 千万不要用 round(pow(10, 9)) + 7。在 C++ 中直接写 const int MOD = 1e9 + 7; 既避免了浮点运算的巨大常数开销, 又杜绝了潜在的精度丢失风险。														
	Book Shop	给定 n 本书, 每本书有对应的价格和页数。总预算最多为 x, 求在预算内能买到的最多总页数。规定每本书最多只能买一次。数据范围: 1 <= n <= 1000, 1 <= x <= 100,000。	1D DP, 0-1 Knapsack (0-1 背包)		1. 状态定义: dp[i] 表示在预算 j 下能买到的最多页数。初始化全为 0。 2. 0-1 背包核心 (防重复拿取): 外层循环按顺序选出每本书。内层循环必须倒序遍历预算 (从 x 降到当前书的价格), 转移方程: dp[i] = max(dp[i], dp[i - price] + pages)。 3. 倒序的物理意义: 算大预算 dp[i] 时, 它依赖的小预算 dp[i-price] 绝对是上一轮的历史干净记录, 还没有被当前这本书碰过。从而在物理上彻底切断了“同一本书被买多次”的可能。	practice w4 q4	1. 局部贪心破坏全局陷阱 (WA): 千万不要用 set.erase() 去物理销毁用过的书。在 DP 里, 不同预算 j 是“平行宇宙”, 小预算用了某本书, 绝不能剥夺大预算使用这本书的权利。 2. 初始化幽灵数据 (WA): 既然题目不要求“恰好花光预算”, 就把 dp 数组全初始化为 0。如果错误地初始化为 -1 且不加判断, -1 加上书的页数会凭空变成一个虚假的合法正数 (如 -1 + 5 = 4), 导致全盘结果被污染。 3. 极性能防 TLE: 双重循环最坏有 1 亿次计算 (1000 * 100000)。正好卡在 1 秒时限边缘。必须在 main 函数开头加上 ios_base::sync_with_stdio(false); cin.tie(NULL); 开启快速 I/O, 否则极易超时。 一、0-1 背包 (0-1 Knapsack) 核心哲学: Take it or leave it (拿或者不拿)。每一件物品都是独一无二的, 且只有一件。 • 状态定义: dp[i] 表示容量为 i 时能获得的最大价值。 • 状态转移: dp[i] = max(dp[i], dp[i - weight[j]] + value[j]) • 黄金铁律: 内层容量循环必须倒序 (从大到小)。 • 为什么倒序? 为了物理切断“时空污染”。算大容量时, 引用的小容量状态必须是上一轮的历史记录, 确保当前物品绝对不被重复累加。 二、完全背包 (Complete Knapsack) 核心哲学: 只要容量够, 同一种物品你想拿多少拿多少。 • 状态定义: dp[i] 表示容量为 i 时能获得的最大价值 (或方案数)。 • 状态转移: dp[i] = max(dp[i], dp[i - weight[j]] + value[j]) (求最值) 或 dp[i] += dp[i - weight[j]] (求方案数)。 • 黄金铁律: 内层容量循环必须正序 (从小到大)。 • 为什么正序? 正是因为我们需要“时空污染”! 当我们在算容量 4 时, 如果用到了容量 2 的结果, 而容量 2 刚好刚刚装入了 2 件物品 A, 那么容量 4 实际上就装入了两件物品 A。正序遍历完美契合了“无限拿取”的物理意义。 完全背包的“神坑”: 排列 vs 组合 就像你在做 Coin Combinations 系列时遇到的: 1. 求组合数 (无序, 2+3 和 3+2 算同一种): • 外层遍历物品, 内层正序遍历容量。 • 逻辑: 硬币按次序出场, 先用 2 填满账本, 再用 3 填满账本。强行规定了拿取的先后顺序, 彻底去重。 2. 求排列数 (有序, 2+3 和 3+2 算两种): • 外层遍历容量, 内层遍历物品。 • 逻辑: 在每一个确定的容量下, 所有硬币一起公平竞争。先拿 2 后拿 3, 和先拿 3 后拿 2 会在不同的容量阶段被重复计算。 三、多重背包 (Multiple Knapsack) 核心哲学: 物品不是无限的, 但也不是只有 1 个, 而是给了你具体的库存限制 (比如物品 A 最多拿 S_k 个)。 • 最朴素解法: 把它当成 0-1 背包。物品 A 有 5 个, 我就当成是 5 个完全独立, 只能拿 1 次的物品 A1, A2, A3, A4, A5。 • 致命缺陷: 如果 S_k 非常大, 时间复杂度会爆炸。 • 进阶优化 (二进制拆分): 不需要真的拆成 S_k 个物品, 而是利用二进制拼凑的原理来打包。假如物品 A 有 13 个, 我们不需要 13 个独立的 A, 我们把它们打包成 4 个包裹, 容量分别是: 1 个、2 个、4 个、6 个 (剩下的)。 你任意 1 到 13 之间的任何一个数量, 都可以用这四个包裹通过 0-1 背包精准凑出来 (比如拿 7 个就是 1+2+4)。这样原本 O(N times K S) 的拆分, 瞬间降维到了 O(N log K S)。 四、分组背包 (Group Knapsack) 核心哲学: 物品被分成了若干个互斥的“门派”, 每个门派里有很多弟子, 但你每次去一个门派, 最多只能挑 1 个弟子带走。 • 状态转移: dp[i] = max(dp[i], dp[i - weight[k]] + value[k]) (其中 S_k 是当前组里的某一个物品)。 • 黄金铁律的嵌套: 1. 最外层循环: 遍历所有的组。 2. 中层循环: 遍历容量 (“必须倒序!” 因为不同组之间是 0-1 背包关系, 不能串味)。 3. 最内层循环: 枚举当前组内的所有物品。 • 为什么把容量放在中间? 保证在同一个容量 S_k 下, 当前组内的所有物品都在互相竞争 (因为中层循环锁定了 S_k, 内层循环轮流用不同的 S_k 去尝试更新 dp[i])。最后留下下来的, 一定是这个组里最能打的那个弟子, 而且完美保证了每组最多只选了一个。														
	Array Description	给定一个长度为 n 的数组, 元素取值范围在 1 到 m 之间, 且相邻两个元素的绝对差最多为 1。数组中有些位置的值未知 (用 0 表示)。求能构造出多少种符合条件的合法数组。结果对 1e9 + 7 取模。数据范围: 1 <= n <= 100,000, 1 <= m <= 100。	2D DP, Sequence DP		1. 状态定义: dp[i][v] 表示长度为 i (前 i 个元素), 且最后一个元素的值为 v 的合法方案数。 2. 逻辑复用魔法: 处理“残缺数组”时, 不要把“当前位置是 0”和“当前位置已知”拆成两截冗长的转移逻辑。通过动态设置当前格子允许填入的数值上下界来统一处理: 如果为 0, 范围是 [1, m]; 如果已知为 x, 范围就是 [x, x]。 3. 状态转移: 在合法的上下界内遍历当前值 curr。因为相邻差值最多为 1, 它只能从上一层的 curr-1, curr, curr+1 三个状态转移而来。即: dp[i][curr] = (dp[i-1][curr-1] + dp[i-1][curr] + dp[i-1][curr+1]) % MOD。	practice w4 q5	1. 越界崩溃陷阱 (RE): 推导 DP 状态用到 i-1 时, 外层循环绝对不能从初始值 0 开始, 否则会直接数组越界 (array[-1]) 导致 Segmentation Fault。必须手动初始化第一阶段, 然后从下一阶段开始循环。 2. 规则忽略陷阱 (WA): 不要以为“当前数字固定已知”就可以直接继承上一层的所有方案。即使当前数字固定, 它也必须遵守“和前一个数字相差不能超过 1”的基本法, 只能继承相邻的三个状态。 3. 极性能陷阱 (TLE): 在执行次数高达上千万次的双重循环最内层, 绝对不要动态创建和销毁 vector 或者复杂对象。系统层面的内存消耗会带来极大的常数开销, 导致本来对的 O(NM) 复杂度直接超时。尽量用最基础的 if 判断和加法。														

QID	Question Name	Des	Tag	Solution	Key Idea	From	Note							
							差分数组防越界核心总结(纯文本归档版) 核心法则: 定义差分数组时, 数组长度必须分配为 $n+1$. 绝不能只分配 n. 原理剖析: 差分法通过修改左右两个端点来实现区间修改。假设对区间 $[L, R]$ 加上数值 V. 标准操作为: 1. 左端点: $diff[L] += V$ 2. 右端点后一位: $diff[R+1] -= V$ 越界触发条件: 当修改区间一直延伸到原数组的最后一位(即 $R = n-1$)时, 右端点的恢复操作会变成 $diff[n] -= V$. 产生后果: 如果数组大小只分配了 n, 其余合法最大索引为 $n-1$. 此时强行访问 $diff[n]$ 会直接导致数组越界。这在C++中会引发严重的内存覆盖(导致程序中其他变量数值被异常篡改, 产生难以调试的玄学Bug), 或导致导致 <code>Runtime Error</code> (段错误崩溃)。 一句话记忆: 区间差分右界必加一, 数组内存务必开 $n+1$。							